

Week_4

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;
    pid_t pid;
    int status;

    printf("Parent Process PID: %d\n", getpid());

    for (i = 1; i <= 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            return 1;
        }

        if (pid == 0)
        {
            printf("Child %d: PID = %d, Parent PID = %d\n", i, getpid(), getppid());
            sleep(i);
            printf("Child %d completed.\n", i);
            exit(0);
        }
    }

    printf("\nParent is waiting using wait()...\n");
    pid = wait(&status);

    if (pid > 0)
    {
        printf("wait(): Child with PID %d completed.\n", pid);
    }

    printf("\nParent is waiting using waitpid()...\n");

    for (i = 0; i < 2; i++)
    {
        pid = waitpid(-1, &status, 0);

        if (pid > 0)
        {
            printf("waitpid(): Child with PID %d completed.\n", pid);
        }
    }

    printf("\nAll child processes completed.\n");

    return 0;
}
```

OUTPUT:

```
root@DESKTOP-QU0HVJI:~/practical/practical-4# gcc wait_waitpid.c -o wait_waitpid
root@DESKTOP-QU0HVJI:~/practical/practical-4# ./wait_waitpid
Parent Process PID: 871
Child 1: PID = 872, Parent PID = 871
Child 2: PID = 873, Parent PID = 871

Parent is waiting using wait()...
Child 3: PID = 874, Parent PID = 871
Child 1 completed.
wait(): Child with PID 872 completed.

Parent is waiting using waitpid()...
Child 2 completed.
waitpid(): Child with PID 873 completed.
Child 3 completed.
waitpid(): Child with PID 874 completed.

All child processes completed.
root@DESKTOP-QU0HVJI:~/practical/practical-4# cat wait_waitpid.c
```

Parent process completed.

REPORT:

I created a parent process that creates multiple child processes using `fork()`. I used `wait()` to wait for a child process to complete and used `waitpid()` to synchronize the remaining child processes. From the output, I understood that `wait()` can wait for any child process, while `waitpid()` gives more control over which child process is waited for.

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());

        printf("Child is exiting...\n");

        exit(0);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is sleeping...\n");
        printf("Child will become a zombie.\n");

        sleep(30);

        printf("Parent completed.\n");
    }

    return 0;
}

```

OUTPUT:

```

root@DESKTOP-QU0HVJI:~/practical/practical-4# nano zombie.c
root@DESKTOP-QU0HVJI:~/practical/practical-4# gcc zombie.c -o zombie
root@DESKTOP-QU0HVJI:~/practical/practical-4# ./zombie
Parent Process
Parent PID: 997
Child PID: 998
Parent is sleeping...
Child will become a zombie.
Child Process
Child PID: 998
Child is exiting...

```

```

GNU nano 7.2 zombie_fixed.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());

        printf("Child is exiting...\n");

        exit(0);
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        // Wait for the child to finish
        wait(NULL);

        printf("Child process completed.\n");
        printf("Zombie process eliminated.\n");
    }

    return 0;
}

```

OUTPUT:

```

root@DESKTOP-QU0HVJJI:~/pratical/practical-4# nano zombie_fixed.c
root@DESKTOP-QU0HVJJI:~/pratical/practical-4# gcc zombie_fixed.c -o zombie_fixed
root@DESKTOP-QU0HVJJI:~/pratical/practical-4# ./zombie_fixed
Parent Process
Parent PID: 1069
Child Process
Child PID: 1070
Child PID: 1070
Child is exiting...
Child process completed.
Zombie process eliminated.
root@DESKTOP-QU0HVJJI:~/pratical/practical-4# |

```

REPORT:

I created a child process that becomes a zombie when the parent does not immediately collect its exit status. I checked the process table and observed the zombie process. Then I modified the program by using proper synchronization with `wait()` so that the parent collects the child process after it finishes. I understood that proper synchronization helps to prevent zombie processes

